

```
// ===== src-tauri/src/commands.rs =====
use crate::config;
use crate::indexer::SessionIndex;
use crate::models::{AppConfig, GitInfo, ProviderInfo, Session, TagsMap, UpdateStats};
use crate::searcher::{self, SearchResult};
use crate::terminal;
use crate::updater::Updater;
use parking_lot::Mutex;
use std::collections::HashMap;
use std::process::Command;
use std::sync::Arc;
use tauri::State;

/// 创建隐藏窗口的 Command (Windows 上不闪 cmd 窗口)
#[cfg(windows)]
fn silent_command(program: &str) -> Command {
    use std::os::windows::process::CommandExt;
    let mut cmd = Command::new(program);
    cmd.creation_flags(0x08000000); // CREATE_NO_WINDOW
    cmd
}

#[cfg(not(windows))]
fn silent_command(program: &str) -> Command {
    Command::new(program)
}

/// 应用全局状态
pub struct AppState {
    pub index: Arc<Mutex<SessionIndex>>,
    pub updater: Arc<Updater>,
    pub config: Arc<Mutex<AppConfig>>,
    /// 缓存的全量会话列表 (供预览等功能查询)
    pub sessions: Arc<Mutex<Vec<Session>>>,
    /// 收藏的会话 ID 列表
    pub favorites: Arc<Mutex<Vec<String>>>,
    /// 会话标签: session_id -> [tag1, tag2, ...]
    pub tags: Arc<Mutex<TagsMap>>,
    /// 会话备注: session_id -> "备注文本"
    pub notes: Arc<Mutex<HashMap<String, String>>>,
    /// 后台扫描是否完成
    pub ready: Arc<std::sync::atomic::AtomicBool>,
}

/// 自动探测默认工作目录: 扫描常见路径, 返回第一个存在且非空的目录
#[tauri::command]
pub fn detect_default_workspace() -> Option<String> {
    let home = dirs::home_dir()?;
    let candidates = if cfg!(windows) {
        vec![
```

```

        std::path::PathBuf::from("D:\\workspace"),
        home.join("workspace"),
        home.join("projects"),
        home.join("code"),
        std::path::PathBuf::from("C:\\dev"),
    ]
} else if cfg!(target_os = "macos") {
    vec![
        home.join("workspace"),
        home.join("projects"),
        home.join("code"),
        home.join("Developer"),
    ]
} else {
    vec![
        home.join("workspace"),
        home.join("projects"),
        home.join("code"),
        home.join("dev"),
    ]
};

candidates
    .into_iter()
    .find(|p| {
        p.is_dir()
        && std::fs::read_dir(p)
            .map(|mut d| d.next().is_some())
            .unwrap_or(false)
    })
    .map(|p| p.to_string_lossy().to_string())
}

/// 打开文件选择对话框（异步，不阻塞 UI）
#[tauri::command]
pub async fn pick_file(app: tauri::AppHandle, title: String) -> Option<String> {
    use tauri::async_runtime;

    async_runtime::spawn_blocking(move || {
        use std::process::Command as StdCommand;

        #[cfg(windows)]
        {
            let script = r#"
                Add-Type -AssemblyName System.Windows.Forms
                $f = New-Object System.Windows.Forms.OpenFileDialog
                $f.Title = 'TITLE'
                $f.Filter = '可执行文件*.exe;*.cmd;*.bat;*.com|所有文件|*.*'
                if ($f.ShowDialog() -eq 'OK') { $f.FileName }
            "#.replace("TITLE", &title);

```

```

let output = StdCommand::new("powershell")
  .args(["-NoProfile", "-Command", &script])
  .output()
  .ok()?;
let path = String::from_utf8_lossy(&output.stdout).trim().to_string();
if path.is_empty() { None } else { Some(path) }
}

#[cfg(target_os = "macos")]
{
  let script = format!(
    "POSIX path of (choose file with prompt \"{ }\\" of type {{\\"public.unix-executable\\" , \\"public.item\\"}}...
    title
  );
  let output = StdCommand::new("osascript")
    .args(["-e", &script])
    .output()
    .ok()?;
  let path = String::from_utf8_lossy(&output.stdout).trim().to_string();
  if path.is_empty() { None } else { Some(path) }
}

#[cfg(target_os = "linux")]
{
  let output = StdCommand::new("zenity")
    .args(["--file-selection", "--title", &title])
    .output()
    .or_else(|_| {
      StdCommand::new("kdialog")
        .args(["--getopenfilename", ".", "**"])
        .output()
    })
    .ok()?;
  let path = String::from_utf8_lossy(&output.stdout).trim().to_string();
  if path.is_empty() { None } else { Some(path) }
}
})
.await
.ok()
.flatten()
}

/// 检查后台数据是否就绪
#[tauri::command]
pub fn is_ready(state: State<AppState>) -> bool {
  state.ready.load(std::sync::atomic::Ordering::Relaxed)
}

/// 全文搜索会话，支持按 provider 过滤
#[tauri::command]
```

```

pub fn search(
    state: State<AppState>,
    query: String,
    provider_filter: Option<String>,
) -> Vec<SearchResult> {
    let max = state.config.lock().ui.max_results;
    let filter = provider_filter.as_deref().filter(|p| *p != "all");
    match state.index.try_lock() {
        Some(index) => searcher::search(&index, &query, max, filter),
        None => Vec::new(),
    }
}

/// 列出会话（按更新时间降序），支持 provider 过滤
#[tauri::command]
pub fn list_sessions(
    state: State<AppState>,
    provider_filter: Option<String>,
) -> Vec<SearchResult> {
    // 后台扫描未完成时直接返回当前索引数据（可能为空）
    if !state.ready.load(std::sync::atomic::Ordering::Relaxed) {
        let index = state.index.lock();
        let filter = provider_filter.as_deref().filter(|p| *p != "all");
        return searcher::list_all(&index, 0, filter);
    }

    let config = state.config.lock().clone();

    // 按需刷新：尝试获取锁，获取不到说明后台正在同步，跳过
    if let Some(index) = state.index.try_lock() {
        let refreshed = state.updater.on_demand_refresh(&index, &config);
        if refreshed {
            // 检测到变化，后台线程刷新（先释放锁再 spawn）
            drop(index);
            let bg_index = Arc::clone(&state.index);
            let bg_sessions = Arc::clone(&state.sessions);
            std::thread::spawn(move || {
                let fresh = crate::scanner::scan_all_sessions();
                let _ = bg_index.lock().rebuild(&fresh);
                *bg_sessions.lock() = fresh;
            });
        } else {
            drop(index);
        }
    }

    // 用 try_lock 获取索引查询，获取不到则返回空（后台正在更新）
    let filter = provider_filter.as_deref().filter(|p| *p != "all");
    match state.index.try_lock() {
        Some(index) => searcher::list_all(&index, config.ui.max_results, filter),
    }
}

```

```
        None => Vec::new(), // 后台正在更新, 前端稍后重试
    }
}

/// 在终端中恢复 AI 编码工具会话
#[tauri::command]
pub fn resume_session(
    state: State<AppState>,
    session_id: String,
    project_path: String,
    provider: String,
) -> Result<(), String> {
    let config = state.config.lock();
    let term = terminal::detect_terminal_with_custom(&config.terminal.preferred, &config.terminal.custom_command);
    terminal::resume_in_terminal(&term, &provider, &project_path, &session_id)
}

/// 在终端中启动新的 CLI 会话
#[tauri::command]
pub fn new_session(
    state: State<AppState>,
    project_path: String,
    provider: String,
) -> Result<(), String> {
    let config = state.config.lock();
    let term = terminal::detect_terminal_with_custom(&config.terminal.preferred, &config.terminal.custom_command);

    // 构建 cd + tool 命令 (无 --resume)
    let cd_cmd = if cfg!(windows) {
        format!("cd /d \"{}\"", project_path)
    } else {
        format!("cd \"{}\"", project_path)
    };
    let tool_cmd = match provider.as_str() {
        "claude" => "claude",
        "codex" => "codex",
        "gemini" => "gemini",
        "opencode" => "opencode",
        "kilo" => "kilo",
        _ => return Err("不支持的工具".to_string()),
    };
    let full_cmd = format!("{}", cd_cmd, tool_cmd);

    let project_name = std::path::Path::new(&project_path)
        .file_name()
        .unwrap_or_default()
        .to_string_lossy();

    let result = match term {
        terminal::TerminalKind::WindowsTerminal => {
```

```

std::process::Command::new("wt.exe")
    .args(["new-tab", "--title", &format!("{}", project_name), "cmd", "/k", &full_cmd])
    .spawn()
}
terminal::TerminalKind::PowerShell => {
    let pwsh = if cfg!(windows) { "pwsh.exe" } else { "pwsh" };
    std::process::Command::new(pwsh)
        .args(["-NoExit", "-Command", &full_cmd])
        .spawn()
}
terminal::TerminalKind::Cmd => {
    std::process::Command::new("cmd.exe")
        .args(["/k", &full_cmd])
        .spawn()
}
#[cfg(target_os = "macos")]
terminal::TerminalKind::MacDefault => {
    let script = format!("tell application \"Terminal\" \"nactivate\ndo script \"{}\" \"nend tell\", full_cmd.repl...
    std::process::Command::new("osascript").args(["-e", &script]).spawn()
}
#[cfg(target_os = "macos")]
terminal::TerminalKind::MacTerm => {
    let script = format!("tell application \"iTerm\" \"nactivate\ntell current window\ncreate tab with default ...
    std::process::Command::new("osascript").args(["-e", &script]).spawn()
}
#[cfg(target_os = "linux")]
terminal::TerminalKind::LinuxTerminal(ref term_name) => {
    let shell_cmd = format!("{}", exec $SHELL, full_cmd);
    std::process::Command::new(term_name).args(["--", "bash", "-c", &shell_cmd]).spawn()
}
_ => {
    return Err("未找到可用终端".to_string());
}
};

result.map(|_| ()).map_err(|e| format!("启动终端失败: {}", e))
}

/// 获取所有已知项目路径（去重，用于新建会话时快速选择）
#[tauri::command]
pub fn get_project_paths(state: State<AppState>) -> Vec<String> {
    let sessions = state.sessions.lock();
    let mut paths: Vec<String> = sessions.iter()
        .map(|s| s.project_path.clone())
        .filter(|p| !p.starts_with("gemini:") && !p.is_empty())
        .collect();
    paths.sort();
    paths.dedup();
    paths
}

```

```
/// 构建 resume 命令字符串 (用于复制到剪贴板)
#[tauri::command]
pub fn copy_command(
    session_id: String,
    project_path: String,
    provider: String,
) -> String {
    terminal::build_resume_command(&provider, &project_path, &session_id)
}

/// 获取更新策略性能统计
#[tauri::command]
pub fn get_stats(state: State<AppState>) -> Vec<UpdateStats> {
    state.updater.get_stats()
}

/// 获取当前配置
#[tauri::command]
pub fn get_config(state: State<AppState>) -> AppConfig {
    state.config.lock().clone()
}

/// 保存配置并更新内存中的配置状态
#[tauri::command]
pub fn save_config(
    state: State<AppState>,
    new_config: AppConfig,
) {
    config::save_config(&new_config);
    *state.config.lock() = new_config;
}

// =====
// 快捷键热更新 — 注销旧快捷键并注册新快捷键
// =====

#[tauri::command]
pub fn update_hotkey(app: tauri::AppHandle, new_hotkey: String) -> Result<(), String> {
    use tauri_plugin_global_shortcut::{GlobalShortcutExt, Shortcut};

    // 注销所有现有快捷键
    let _ = app.global_shortcut().unregister_all();

    // 解析新快捷键字符串 (如 "Ctrl+Shift+C")
    let shortcut: Shortcut = new_hotkey.parse().map_err(|e| format!("无效快捷键: {:?}", e))?;

    // 注册新快捷键并绑定 toggle_window 处理
    app.global_shortcut().on_shortcut(shortcut, |app, _shortcut, event| {
        if event.state() == tauri_plugin_global_shortcut::ShortcutState::Pressed {
```

```

        crate::toggle_window(app);
    }
    }).map_err(|e| format!("注册失败: {}", e))?;

    Ok(())
}

// =====
// Feature 1: 会话预览 — 返回指定会话的最后 3 条用户消息
// =====

#[tauri::command]
pub fn get_session_preview(
    state: State<AppState>,
    session_id: String,
) -> Vec<String> {
    let sessions = state.sessions.lock();
    sessions
        .iter()
        .find(|s| s.session_id == session_id)
        .map(|s| {
            let msgs = &s.user_messages;
            let start = if msgs.len() > 3 { msgs.len() - 3 } else { 0 };
            msgs[start..].to_vec()
        })
        .unwrap_or_default()
}

// =====
// Feature 2: 项目 Git 信息
// =====

#[tauri::command]
pub async fn get_project_git_info(project_path: String) -> Option<GitInfo> {
    tauri::async_runtime::spawn_blocking(move || {
        // 获取当前分支名
        let branch_output = silent_command("git")
            .args(["-C", &project_path, "rev-parse", "--abbrev-ref", "HEAD"])
            .output()
            .ok()?;
        if !branch_output.status.success() {
            return None;
        }
        let branch = String::from_utf8_lossy(&branch_output.stdout)
            .trim()
            .to_string();

        // 获取未提交变更数
        let status_output = silent_command("git")
            .args(["-C", &project_path, "status", "--porcelain"])

```



```
.output()
.ok()?;
let dirty_count = String::from_utf8_lossy(&status_output.stdout)
    .lines()
    .filter(|l| !l.is_empty())
    .count() as u32;

Some(GitInfo {
    branch,
    dirty_count,
})
})
.await
.ok()
.flatten()
}

// =====
// Feature 3: 收藏/置顶
// =====

/// 收藏文件路径
fn favorites_path() -> std::path::PathBuf {
    config::retalk_dir().join("favorites.json")
}

/// 从磁盘加载收藏列表
pub fn load_favorites() -> Vec<String> {
    let path = favorites_path();
    if path.exists() {
        std::fs::read_to_string(&path)
            .ok()
            .and_then(|c| serde_json::from_str(&c).ok())
            .unwrap_or_default()
    } else {
        Vec::new()
    }
}

/// 保存收藏列表到磁盘
fn save_favorites(favs: &[String]) {
    let path = favorites_path();
    if let Some(parent) = path.parent() {
        let _ = std::fs::create_dir_all(parent);
    }
    let _ = std::fs::write(&path, serde_json::to_string_pretty(favs).unwrap_or_default());
}

#[tauri::command]
pub fn toggle_favorite(
```

```

    state: State<AppState>,
    session_id: String,
) -> bool {
    let mut favs = state.favorites.lock();
    let is_fav = if let Some(pos) = favs.iter().position(|id| id == &session_id) {
        favs.remove(pos);
        false
    } else {
        favs.push(session_id);
        true
    };
    save_favorites(&favs);
    is_fav
}

#[tauri::command]
pub fn get_favorites(state: State<AppState>) -> Vec<String> {
    state.favorites.lock().clone()
}

// =====
// 会话导出 — 导出为 Markdown
// =====

#[tauri::command]
pub fn export_session_markdown(
    state: State<AppState>,
    session_id: String,
) -> Result<String, String> {
    let sessions = state.sessions.lock();
    let session = sessions
        .iter()
        .find(|s| s.session_id == session_id)
        .ok_or("会话未找到");

    let mut md = format!("# {} - {}\n", session.project_name, session.provider);
    md += &format!("***项目路径:** {}\n", session.project_path);
    md += &format!("***会话ID:** {}\n", session.session_id);
    md += &format!("***消息数:** {}\n", session.message_count);
    if session.total_tokens > 0 {
        md += &format!("***Token 数:** {}\n", session.total_tokens);
    }
    md += "\n---\n";

    for (i, msg) in session.user_messages.iter().enumerate() {
        md += &format!("### 消息 {}{}\n{}\n", i + 1, msg);
    }

    Ok(md)
}

```

```
#[tauri::command]
pub fn export_session_to_file(
    state: State<AppState>,
    session_id: String,
    file_path: String,
) -> Result<(), String> {
    let md = export_session_markdown_inner(&state, &session_id)?;
    std::fs::write(&file_path, md).map_err(|e| format!("写入文件失败: {}", e))
}

/// 内部复用: 生成 Markdown 文本
fn export_session_markdown_inner(
    state: &State<AppState>,
    session_id: &str,
) -> Result<String, String> {
    let sessions = state.sessions.lock();
    let session = sessions
        .iter()
        .find(|s| s.session_id == session_id)
        .ok_or("会话未找到");

    let mut md = format!("# {} - {}\n\n", session.project_name, session.provider);
    md += &format!("***项目路径:** {}\n", session.project_path);
    md += &format!("***会话ID:** {}\n", session.session_id);
    md += &format!("***消息数:** {}\n", session.message_count);
    if session.total_tokens > 0 {
        md += &format!("***Token 数:** {}\n", session.total_tokens);
    }
    md += "\n---\n\n";

    for (i, msg) in session.user_messages.iter().enumerate() {
        md += &format!("### 消息 {}{}\n{}\n", i + 1, msg);
    }

    Ok(md)
}

/// 获取用户桌面路径
#[tauri::command]
pub fn get_desktop_path() -> Result<String, String> {
    dirs::desktop_dir()
        .map(|p| p.to_string_lossy().to_string())
        .ok_or_else(|| "无法获取桌面路径".to_string())
}

// =====
// Feature 4: 快捷操作 — 在 VS Code / 文件管理器中打开
// =====
```

```

/// 获取 VS Code 命令名 (Windows 用 code.cmd 避免闪 cmd 窗口)
fn code_cmd() -> &'static str {
    if cfg!(windows) { "code.cmd" } else { "code" }
}

#[tauri::command]
pub fn open_in_vscode(project_path: String) -> Result<(), String> {
    silent_command(code_cmd())
        .arg(&project_path)
        .spawn()
        .map(|_| ())
        .map_err(|e| e.to_string())
}

/// 在系统文件管理器中打开目录 (跨平台)
#[tauri::command]
pub fn open_in_explorer(project_path: String) -> Result<(), String> {
    #[cfg(windows)]
    {
        Command::new("explorer")
            .arg(&project_path)
            .spawn()
            .map(|_| ())
            .map_err(|e| e.to_string())
    }
    #[cfg(target_os = "macos")]
    {
        Command::new("open")
            .arg(&project_path)
            .spawn()
            .map(|_| ())
            .map_err(|e| e.to_string())
    }
    #[cfg(target_os = "linux")]
    {
        Command::new("xdg-open")
            .arg(&project_path)
            .spawn()
            .map(|_| ())
            .map_err(|e| e.to_string())
    }
}

/// 在文件管理器中打开并选中指定文件 (跨平台)
#[tauri::command]
pub fn open_in_explorer_select(file_path: String) -> Result<(), String> {
    #[cfg(windows)]
    {
        Command::new("explorer")
            .args(["/select,", &file_path])
    }

```

```
.spawn()
.map(|_| ())
.map_err(|e| e.to_string())
}
#[cfg(target_os = "macos")]
{
    // macOS: open -R 可以在 Finder 中选中文件
    Command::new("open")
        .args(["-R", &file_path])
        .spawn()
        .map(|_| ())
        .map_err(|e| e.to_string())
}
#[cfg(target_os = "linux")]
{
    // Linux: xdg-open 不支持选中文件, 打开其所在目录
    let dir = std::path::Path::new(&file_path)
        .parent()
        .unwrap_or(std::path::Path::new("/"));
    Command::new("xdg-open")
        .arg(dir)
        .spawn()
        .map(|_| ())
        .map_err(|e| e.to_string())
}
}

// =====
// Feature 6: 会话标签系统
// =====

/// 标签文件路径
fn tags_path() -> std::path::PathBuf {
    config::retalk_dir().join("tags.json")
}

/// 从磁盘加载标签
pub fn load_tags() -> TagsMap {
    let path = tags_path();
    if path.exists() {
        std::fs::read_to_string(&path)
            .ok()
            .and_then(|c| serde_json::from_str(&c).ok())
            .unwrap_or_default()
    } else {
        HashMap::new()
    }
}

/// 保存标签到磁盘
```

```
fn save_tags(tags: &TagsMap) {
    let path = tags_path();
    if let Some(parent) = path.parent() {
        let _ = std::fs::create_dir_all(parent);
    }
    let _ = std::fs::write(&path, serde_json::to_string_pretty(tags).unwrap_or_default());
}

#[tauri::command]
pub fn set_tags(
    state: State<AppState>,
    session_id: String,
    tags: Vec<String>,
) {
    let mut all_tags = state.tags.lock();
    if tags.is_empty() {
        all_tags.remove(&session_id);
    } else {
        all_tags.insert(session_id, tags);
    }
    save_tags(&all_tags);
}

#[tauri::command]
pub fn get_all_tags(state: State<AppState>) -> TagsMap {
    state.tags.lock().clone()
}

// =====
// 会话备注 — 独立存储，不修改原始对话文件
// =====

fn notes_path() -> std::path::PathBuf {
    config::retalk_dir().join("notes.json")
}

pub fn load_notes() -> HashMap<String, String> {
    let path = notes_path();
    if path.exists() {
        std::fs::read_to_string(&path)
            .ok()
            .and_then(|c| serde_json::from_str(&c).ok())
            .unwrap_or_default()
    } else {
        HashMap::new()
    }
}

fn save_notes(notes: &HashMap<String, String>) {
    let path = notes_path();
```

```
    if let Some(parent) = path.parent() {
        let _ = std::fs::create_dir_all(parent);
    }
    let _ = std::fs::write(&path, serde_json::to_string_pretty(&notes).unwrap_or_default());
}

#[tauri::command]
pub fn set_note(state: State<AppState>, session_id: String, note: String) {
    let mut notes = state.notes.lock();
    if note.trim().is_empty() {
        notes.remove(&session_id);
    } else {
        notes.insert(session_id, note);
    }
    save_notes(&notes);
}

#[tauri::command]
pub fn get_all_notes(state: State<AppState>) -> HashMap<String, String> {
    state.notes.lock().clone()
}

// =====
// 重建索引
// =====

/// 触发后台重建索引 (非阻塞)
#[tauri::command]
pub fn rebuild_index(state: State<AppState>) {
    let index = Arc::clone(&state.index);
    let sessions_cache = Arc::clone(&state.sessions);
    std::thread::spawn(move || {
        let sessions = crate::scanner::scan_all_sessions();
        let _ = index.lock().rebuild(&sessions);
        *sessions_cache.lock() = sessions;
        eprintln!("[retalk] 索引重建完成");
    });
}

// =====
// Feature 1: 空状态引导 — 返回各 provider 的可用状态
// =====

#[tauri::command]
pub fn get_provider_status() -> Vec<ProviderInfo> {
    use crate::providers;
    let all: Vec<Box<dyn providers::SessionProvider>> = vec![
        Box::new(providers::claude::ClaudeProvider),
        Box::new(providers::codex::CodexProvider),
        Box::new(providers::gemini::GeminiProvider),
    ]
}
```

```

Box::new(providers::opencode::OpenCodeProvider),
Box::new(providers::kilo::KiloProvider),
];
all.iter()
.map(|p| ProviderInfo {
    name: p.name().to_string(),
    available: p.is_available(),
})
.collect()
}

// =====
// Feature 6: 批量导出 — 合并多个会话为 Markdown
// =====

#[tauri::command]
pub fn batch_export_markdown(
    state: State<AppState>,
    session_ids: Vec<String>,
) -> Result<String, String> {
    let sessions = state.sessions.lock();
    let mut md = String::new();

    for sid in &session_ids {
        if let Some(session) = sessions.iter().find(|s| s.session_id == *sid) {
            md += &format!("{}", session.project_name, session.provider);
            md += &format!("{}", session.project_path);
            md += &format!("{}", session.session_id);
            md += &format!("{}", session.message_count);
            if session.total_tokens > 0 {
                md += &format!("{}", session.total_tokens);
            }
            md += "\n---\n";
            for (i, msg) in session.user_messages.iter().enumerate() {
                md += &format!("{}", i + 1, msg);
            }
            md += "\n---\n";
        }
    }

    if md.is_empty() {
        return Err("未找到任何匹配的会话".to_string());
    }
    Ok(md)
}

// =====
// Feature 7: 自动标签 — 根据首条消息关键词自动打标
// =====

```



```
#[tauri::command]
pub fn auto_tag_sessions(state: State<AppState>) -> u32 {
    let sessions = state.sessions.lock();
    let mut tags = state.tags.lock();
    let mut count = 0;

    for session in sessions.iter() {
        // 已有标签的跳过
        if tags.contains_key(&session.session_id) {
            continue;
        }

        let text = session.first_prompt.to_lowercase();
        let mut auto_tags = Vec::new();

        if text.contains("bug")
            || text.contains("fix")
            || text.contains("error")
            || text.contains("修复")
            || text.contains("报错")
        {
            auto_tags.push("bug修复".to_string());
        }
        if text.contains("refactor")
            || text.contains("重构")
            || text.contains("优化")
            || text.contains("cleanup")
        {
            auto_tags.push("重构".to_string());
        }
        if text.contains("add")
            || text.contains("new")
            || text.contains("feature")
            || text.contains("新增")
            || text.contains("添加")
            || text.contains("实现")
        {
            auto_tags.push("新功能".to_string());
        }
        if text.contains("test") || text.contains("测试") {
            auto_tags.push("测试".to_string());
        }
        if text.contains("deploy")
            || text.contains("部署")
            || text.contains("build")
            || text.contains("打包")
        {
            auto_tags.push("部署".to_string());
        }
        if text.contains("doc") || text.contains("文档") || text.contains("readme") {
```

```

        auto_tags.push("文档".to_string());
    }

    if !auto_tags.is_empty() {
        tags.insert(session.session_id.clone(), auto_tags);
        count += 1;
    }
}

save_tags(&tags);
count
}

// =====
// 生态面板 — 跨工具 Skills/MCP/配置 扫描与管理
// =====

#[tauri::command]
pub fn get_ecosystem() -> crate::ecosystem::EcosystemData {
    crate::ecosystem::scan_ecosystem()
}

/// 获取 npm 命令名 (Windows 用 npm.cmd)
fn npm_cmd() -> &'static str {
    if cfg!(windows) { "npm.cmd" } else { "npm" }
}

/// 检查单个 CLI 工具的最新版本 (通过 npm view, 异步避免阻塞 UI)
#[tauri::command]
pub async fn check_tool_update(tool: String) -> Result<serde_json::Value, String> {
    let npm_pkg = match tool.as_str() {
        "claude" => "@anthropic-ai/claude-code",
        "codex" => "@openai/codex",
        "gemini" => "@google/gemini-cli",
        "opencode" => "opencode-ai",
        _ => return Err("不支持更新检查".to_string()),
    };
    let npm_pkg = npm_pkg.to_string();

    tauri::async_runtime::spawn_blocking(move || {
        let output = silent_command(npm_cmd())
            .args(["view", &npm_pkg, "version"])
            .output()
            .map_err(|e| format!("npm 执行失败: {}", e))?;

        if output.status.success() {
            let latest = String::from_utf8_lossy(&output.stdout).trim().to_string();
            Ok(serde_json::json!({ "tool": tool, "latest": latest, "package": npm_pkg }))
        } else {
            Err("无法获取最新版本".to_string())
        }
    })
}

```

```

    }
  })
  .await
  .map_err(|e| format!("任务失败: {}", e))?
}

/// 安装 CLI 工具 (npm install -g, 异步避免阻塞 UI)
#[tauri::command]
pub async fn install_cli_tool(pkg: String) -> Result<String, String> {
  tauri::async_runtime::spawn_blocking(move || {
    let output = silent_command(npm_cmd())
      .args(["install", "-g", &pkg])
      .output()
      .map_err(|e| format!("npm 执行失败: {}", e))?;
    if output.status.success() {
      Ok(format!("{}", 安装成功, pkg))
    } else {
      let err = String::from_utf8_lossy(&output.stderr).to_string();
      Err(format!("{}", 安装失败: {}, err))
    }
  })
  .await
  .map_err(|e| format!("任务失败: {}", e))?
}

#[tauri::command]
pub fn toggle_mcp_server(server_name: String, source: String, enabled: bool) -> Result<(), String> {
  crate::ecosystem::toggle_mcp_in_file(&source, &server_name, enabled)
}

// =====
// 插件管理 — 调用 claude plugins CLI
// =====

#[tauri::command]
pub async fn plugin_toggle(plugin_id: String, enabled: bool) -> Result<String, String> {
  tauri::async_runtime::spawn_blocking(move || {
    let action = if enabled { "enable" } else { "disable" };
    let output = silent_command("claude")
      .args(["plugins", action, &plugin_id])
      .output()
      .map_err(|e| format!("执行失败: {}", e))?;
    if output.status.success() {
      Ok(format!("{}", 已{}, plugin_id, if enabled { "启用" } else { "禁用" })))
    } else {
      let err = String::from_utf8_lossy(&output.stderr).to_string();
      Err(format!("{}", 操作失败: {}, err))
    }
  })
  .await

```

```

        .map_err(|e| format!("任务失败: {}", e))?
    }

#[tauri::command]
pub async fn plugin_uninstall(plugin_id: String) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || {
        let output = silent_command("claude")
            .args(["plugins", "uninstall", &plugin_id])
            .output()
            .map_err(|e| format!("执行失败: {}", e))?;
        if output.status.success() {
            Ok(format!("{}", "已卸载", plugin_id))
        } else {
            let err = String::from_utf8_lossy(&output.stderr).to_string();
            Err(format!("卸载失败: {}", err))
        }
    })
    .await
    .map_err(|e| format!("任务失败: {}", e))?
}

/// 安装插件 (调用 claude plugins install CLI, 异步避免阻塞 UI)
#[tauri::command]
pub async fn plugin_install(plugin_id: String) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || {
        let output = silent_command("claude")
            .args(["plugins", "install", &plugin_id])
            .output()
            .map_err(|e| format!("执行失败: {}", e))?;
        if output.status.success() {
            Ok(format!("{}", "已安装", plugin_id))
        } else {
            let err = String::from_utf8_lossy(&output.stderr).to_string();
            Err(format!("安装失败: {}", err))
        }
    })
    .await
    .map_err(|e| format!("任务失败: {}", e))?
}

/// 添加 MCP 服务器到 Claude Code settings.json
#[tauri::command]
pub fn add_mcp_server_cmd(name: String, command: String, args: Vec<String>) -> Result<(), String> {
    crate::ecosystem::add_mcp_server(&name, &command, &args)
}

/// 移除 MCP 服务器 (通用: 根据 tool 分发到 CLI 或文件删除, 异步)
#[tauri::command]
pub async fn remove_mcp_server(tool: String, name: String, source: String) -> Result<String, String> {
    match tool.as_str() {

```

```

        "codex" => codex_mcp_remove(name).await,
        _ => {
            // 从配置文件 (settings.json / .mcp.json) 中移除 (文件 IO, 用 spawn_blocking 包裹)
            tauri::async_runtime::spawn_blocking(move || {
                crate::ecosystem::remove_mcp_from_file(&source, &name)?;
                Ok(format!("{}", 已移除", name))
            })
            .await
            .map_err(|e| format!("任务失败: {}", e))?
        }
    }
}

// =====
// Codex MCP 管理 — 调用 codex mcp CLI
// =====

/// 添加 Codex MCP 服务器 (调用 codex mcp add CLI, 异步避免阻塞 UI)
#[tauri::command]
pub async fn codex_mcp_add(name: String, command: String, args: Vec<String>) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || {
        let mut cmd_args = vec!["mcp".to_string(), "add".to_string(), name.clone(), "--".to_string()];
        cmd_args.push(command);
        cmd_args.extend(args);
        let output = silent_command("codex")
            .args(&cmd_args)
            .output()
            .map_err(|e| format!("执行失败: {}", e))?;
        if output.status.success() {
            Ok(format!("Codex MCP {} 已添加", name))
        } else {
            Err(String::from_utf8_lossy(&output.stderr).to_string())
        }
    })
    .await
    .map_err(|e| format!("任务失败: {}", e))?
}

/// 移除 Codex MCP 服务器 (调用 codex mcp remove CLI, 异步避免阻塞 UI)
#[tauri::command]
pub async fn codex_mcp_remove(name: String) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || {
        let output = silent_command("codex")
            .args(["mcp", "remove", &name])
            .output()
            .map_err(|e| format!("执行失败: {}", e))?;
        if output.status.success() {
            Ok(format!("Codex MCP {} 已移除", name))
        } else {
            Err(String::from_utf8_lossy(&output.stderr).to_string())
        }
    })
    .await
    .map_err(|e| format!("任务失败: {}", e))?
}

```

```

    }
  })
  .await
  .map_err(|e| format!("任务失败: {}", e))?
}

// =====
// Gemini 扩展管理 — 调用 gemini extensions CLI
// =====

/// 安装 Gemini 扩展 (异步避免阻塞 UI)
#[tauri::command]
pub async fn gemini_ext_install(source: String) -> Result<String, String> {
  tauri::async_runtime::spawn_blocking(move || {
    let output = silent_command("gemini")
      .args(["extensions", "install", &source])
      .output()
      .map_err(|e| format!("执行失败: {}", e))?;
    if output.status.success() {
      Ok("Gemini 扩展已安装".to_string())
    } else {
      Err(String::from_utf8_lossy(&output.stderr).to_string())
    }
  })
  .await
  .map_err(|e| format!("任务失败: {}", e))?
}

/// 启用/禁用 Gemini 扩展 (异步避免阻塞 UI)
#[tauri::command]
pub async fn gemini_ext_toggle(name: String, enabled: bool) -> Result<String, String> {
  tauri::async_runtime::spawn_blocking(move || {
    let action = if enabled { "enable" } else { "disable" };
    let output = silent_command("gemini")
      .args(["extensions", action, &name])
      .output()
      .map_err(|e| format!("执行失败: {}", e))?;
    if output.status.success() {
      Ok(format!("{}", name, if enabled { "启用" } else { "禁用" })))
    } else {
      Err(String::from_utf8_lossy(&output.stderr).to_string())
    }
  })
  .await
  .map_err(|e| format!("任务失败: {}", e))?
}

/// 卸载 Gemini 扩展 (异步避免阻塞 UI)
#[tauri::command]
pub async fn gemini_ext_uninstall(name: String) -> Result<String, String> {

```

```
tauri::async_runtime::spawn_blocking(move || {
    let output = silent_command("gemini")
        .args(["extensions", "uninstall", &name])
        .output()
        .map_err(|e| format!("执行失败: {}", e))?;
    if output.status.success() {
        Ok(format!("{}", 已卸载", name))
    } else {
        Err(String::from_utf8_lossy(&output.stderr).to_string())
    }
})
.await
.map_err(|e| format!("任务失败: {}", e))?
}

#[tauri::command]
pub async fn plugin_update(plugin_id: String) -> Result<String, String> {
    tauri::async_runtime::spawn_blocking(move || {
        let output = silent_command("claude")
            .args(["plugins", "update", &plugin_id])
            .output()
            .map_err(|e| format!("执行失败: {}", e))?;
        if output.status.success() {
            Ok(format!("{}", 已更新", plugin_id))
        } else {
            let err = String::from_utf8_lossy(&output.stderr).to_string();
            Err(format!("更新失败: {}", err))
        }
    })
    .await
    .map_err(|e| format!("任务失败: {}", e))?
}

// =====
// Feature 8: 开机自启 (跨平台)
// =====

/// 设置开机自启 (跨平台, 异步避免注册表/文件 IO 阻塞 UI)
#[tauri::command]
pub async fn set_autostart(enabled: bool) -> Result<(), String> {
    tauri::async_runtime::spawn_blocking(move || set_autostart_impl(enabled))
        .await
        .map_err(|e| format!("任务失败: {}", e))?
}

/// Windows: 通过注册表设置开机自启
#[cfg(windows)]
fn set_autostart_impl(enabled: bool) -> Result<(), String> {
    let exe = std::env::current_exe().map_err(|e| e.to_string())?;
    let exe_path = exe.to_string_lossy().to_string();
```

```

if enabled {
    silent_command("reg")
    .args([
        "add",
        "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        "/v",
        "retalk",
        "/t",
        "REG_SZ",
        "/d",
        &exe_path,
        "/f",
    ])
    .output()
    .map_err(|e| e.to_string());
} else {
    silent_command("reg")
    .args([
        "delete",
        "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        "/v",
        "retalk",
        "/f",
    ])
    .output()
    .map_err(|e| e.to_string());
}
Ok(())
}

/// macOS: 通过 LaunchAgents plist 设置开机自启
#[cfg(target_os = "macos")]
fn set_autostart_impl(enabled: bool) -> Result<(), String> {
    let plist_dir = dirs::home_dir()
        .ok_or("无法获取 home 目录")?
        .join("Library")
        .join("LaunchAgents");
    let plist_path = plist_dir.join("com.retalk.app.plist");

    if enabled {
        let exe = std::env::current_exe().map_err(|e| e.to_string());
        let plist = format!(
            r#"<?xml version="1.0" encoding="UTF-8">
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
<key>Label</key>
<string>com.retalk.app</string>
<key>ProgramArguments</key>

```



```
<array>
  <string>{}</string>
</array>
<key>RunAtLoad</key>
<true/>
</dict>
</plist>"#,
    exe.to_string_lossy()
);
std::fs::create_dir_all(&plist_dir).map_err(|e| e.to_string())?;
std::fs::write(&plist_path, plist).map_err(|e| e.to_string())?;
} else {
    let _ = std::fs::remove_file(&plist_path);
}
Ok(())
}

/// Linux: 通过 XDG autostart desktop 文件设置开机自启
#[cfg(target_os = "linux")]
fn set_autostart_impl(enabled: bool) -> Result<(), String> {
    let autostart_dir = dirs::config_dir()
        .ok_or("无法获取配置目录")?
        .join("autostart");
    let desktop_path = autostart_dir.join("retalk.desktop");

    if enabled {
        let exe = std::env::current_exe().map_err(|e| e.to_string())?;
        let desktop = format!(
            "[Desktop Entry]\nType=Application\nName=retalk\nExec={}\nX-GNOME-Autostart-enabled=true\n",
            exe.to_string_lossy()
        );
        std::fs::create_dir_all(&autostart_dir).map_err(|e| e.to_string())?;
        std::fs::write(&desktop_path, desktop).map_err(|e| e.to_string())?;
    } else {
        let _ = std::fs::remove_file(&desktop_path);
    }
    Ok(())
}

/// 查询开机自启状态 (跨平台, 异步避免注册表查询阻塞 UI)
#[tauri::command]
pub async fn get_autostart() -> bool {
    tauri::async_runtime::spawn_blocking(get_autostart_impl)
        .await
        .unwrap_or(false)
}

#[cfg(windows)]
fn get_autostart_impl() -> bool {
    Command::new("reg")
```

```
.args([
  "query",
  "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run",
  "/v",
  "retalk",
])
.output()
.map(|o| o.status.success())
.unwrap_or(false)
}

#[cfg(target_os = "macos")]
fn get_autostart_impl() -> bool {
  dirs::home_dir()
    .map(|h| h.join("Library/LaunchAgents/com.retalk.app.plist").exists())
    .unwrap_or(false)
}

#[cfg(target_os = "linux")]
fn get_autostart_impl() -> bool {
  dirs::config_dir()
    .map(|c| c.join("autostart/retalk.desktop").exists())
    .unwrap_or(false)
}

// ===== src-tauri/src/config.rs =====
use crate::models::AppConfig;
use std::fs;
use std::path::PathBuf;

/// 获取 retalk 数据目录: ~/.claude/retalk/
pub fn retalk_dir() -> PathBuf {
  dirs::home_dir()
    .expect("无法获取 home 目录")
    .join(".claude")
    .join("retalk")
}

/// 获取 Claude Code 数据目录: ~/.claude/
pub fn claude_dir() -> PathBuf {
  dirs::home_dir()
    .expect("无法获取 home 目录")
    .join(".claude")
}

/// 配置文件路径
fn config_path() -> PathBuf {
  retalk_dir().join("config.toml")
}
```

```

/// 加载配置, 不存在则创建默认配置
pub fn load_config() -> AppConfig {
    let path = config_path();
    if path.exists() {
        let content = fs::read_to_string(&path).unwrap_or_default();
        toml::from_str(&content).unwrap_or_default()
    } else {
        let config = AppConfig::default();
        save_config(&config);
        config
    }
}

/// 保存配置到文件
pub fn save_config(config: &AppConfig) {
    let path = config_path();
    if let Some(parent) = path.parent() {
        let _ = fs::create_dir_all(parent);
    }
    let content = toml::to_string_pretty(config).expect("序列化配置失败");
    let _ = fs::write(&path, content);
}

// ===== src-tauri/src/ecosystem.rs =====
use serde::Serialize;
use std::fs;
use std::path::PathBuf;

#[derive(Debug, Clone, Serialize)]
pub struct EcosystemData {
    pub skills: Vec<SkillInfo>,
    pub mcp_servers: Vec<McpServerInfo>,
    pub overview: Vec<ToolOverview>,
    pub plugins: Vec<PluginInfo>,
    pub available_plugins: Vec<AvailablePlugin>,
    pub extensions: Vec<ExtensionInfo>,
}

#[derive(Debug, Clone, Serialize)]
pub struct ToolOverview {
    pub name: String,
    pub installed: bool,
    pub version: String,
    pub data_dir: String,
    pub session_count: u32,
    pub mcp_count: u32,
    pub skill_count: u32,
}

#[derive(Debug, Clone, Serialize)]

```

```
pub struct ExtensionInfo {
    pub tool: String,    // "gemini"
    pub name: String,
    pub description: String,
    pub enabled: bool,
    pub source: String,  // 安装来源 (git URL 或本地路径)
}
```

```
#[derive(Debug, Clone, Serialize)]
pub struct AvailablePlugin {
    pub name: String,
    pub marketplace: String,
    pub description: String,
    /// 完整标识符: name@marketplace, 用于安装命令
    pub full_id: String,
}
```

```
#[derive(Debug, Clone, Serialize)]
pub struct PluginInfo {
    pub name: String,
    pub marketplace: String,
    pub version: String,
    pub description: String,
    pub scope: String,
    pub installed_at: String,
    pub install_path: String,
    pub has_skills: bool,
    pub has_mcp: bool,
    pub skill_count: u32,
    pub enabled: bool,
    /// 完整标识符: name@marketplace
    pub full_id: String,
}
```

```
#[derive(Debug, Clone, Serialize)]
pub struct SkillInfo {
    pub tool: String,
    pub name: String,
    pub description: String,
    pub path: String,
    pub scope: String,
}
```

```
#[derive(Debug, Clone, Serialize)]
pub struct McpServerInfo {
    pub tool: String,
    pub name: String,
    pub command: String,
    pub args: Vec<String>,
    pub enabled: bool,
}
```

```
pub source: String,
}

/// 扫描所有 AI 编码工具的生态信息
pub fn scan_ecosystem() -> EcosystemData {
  let mut skills = Vec::new();
  let mut mcp_servers = Vec::new();

  // Claude Code
  scan_claude_skills(&mut skills);
  scan_claude_mcp(&mut mcp_servers);

  // Codex CLI
  scan_codex_skills(&mut skills);
  scan_codex_mcp(&mut mcp_servers);

  // Gemini CLI
  scan_gemini_mcp(&mut mcp_servers);

  // OpenCode
  scan_opencode_mcp(&mut mcp_servers);

  // Kilo Code
  scan_kilo_mcp(&mut mcp_servers);

  // Claude Code + Codex CLI 插件
  let mut plugins = scan_claude_plugins();
  plugins.extend(scan_codex_plugins());
  let available_plugins = scan_available_plugins(&plugins);

  // Gemini 扩展
  let extensions = scan_gemini_extensions();

  // 工具概览
  let overview = build_overview(&skills, &mcp_servers, &plugins, &extensions);

  EcosystemData {
    skills,
    mcp_servers,
    overview,
    plugins,
    available_plugins,
    extensions,
  }
}

/// 构建各工具概览信息
fn build_overview(
  skills: &[SkillInfo],
  mcp_servers: &[McpServerInfo],
```

```
_plugins: &[PluginInfo],
_extensions: &[ExtensionInfo],
) -> Vec<ToolOverview> {
  use std::process::Command;
  let home = dirs::home_dir().unwrap_or_default();

  let tools = vec![
    ("Claude Code", "claude", home.join(".claude")),
    ("Codex CLI", "codex", home.join(".codex")),
    ("Gemini CLI", "gemini", home.join(".gemini")),
    ("OpenCode", "opencode", home.join(".local").join("share").join("opencode")),
    ("Kilo Code", "kilo", home.join(".local").join("share").join("kilo")),
  ];

  tools.iter().map(|(display_name, cmd, data_dir)| {
    // 检测是否安装并获取版本
    let (installed, version) = get_tool_version(cmd);

    // 统计数量
    let mcp_count = mcp_servers.iter().filter(|s| s.tool == *cmd).count() as u32;
    let skill_count = if *cmd == "claude" {
      skills.len() as u32
    } else {
      0
    };

    // 统计会话数 (检查数据目录大小/文件数)
    let session_count = if *cmd == "claude" {
      count_claude_sessions(&home)
    } else if data_dir.exists() {
      count_files_in_dir(data_dir, "jsonl") + count_files_in_dir(data_dir, "json")
    } else {
      0
    };

    ToolOverview {
      name: display_name.to_string(),
      installed,
      version,
      data_dir: data_dir.to_string_lossy().to_string(),
      session_count,
      mcp_count,
      skill_count,
    }
  }).collect()
}

fn get_tool_version(cmd: &str) -> (bool, String) {
  #[cfg(windows)]
  fn make_cmd(program: &str) -> std::process::Command {
```